

Caso Práctico: Gestión de infraestructura como código (IaC) y ambientes de nube AWS

Instrucciones generales

Este caso debe resolverse de forma individual y asíncrona. No hay una única respuesta correcta — lo que evaluamos es tu **razonamiento técnico, criterio de diseño y capacidad de justificar decisiones** bajo restricciones reales.

Entrega tu solución en el formato que prefieras: documento, repositorio Git, diagramas, o combinación de estos. En la entrevista posterior revisaremos juntos lo que entregaste.

Tiempo estimado: 3 a 4 horas

Ejes de evaluación transversales a todas las tareas:

Eje	Qué buscamos
💰 Costo-eficiencia	Decisiones que optimicen gasto sin sacrificar SLA
🔒 Seguridad	Least privilege, encriptación, auditoría, zero-trust
⚙️ Operación	IaC, automatización, reducción de toil manual
📡 Observabilidad	Alertas accionables, trazabilidad, dashboards útiles

Contexto de negocio

Deuna! es una plataforma fintech de alto volumen transaccional que opera sobre AWS con arquitectura de microservicios. Procesa dos categorías de transacciones:

- **Monetarias:** recargas, pagos, transferencias, retiros
- **No monetarias:** onboarding, actualizaciones de perfil, consultas de saldo

La infraestructura incluye clústeres ECS, bases de datos Aurora PostgreSQL, almacenamiento S3, y una herramienta de observabilidad externa que ingesta CloudWatch Logs. El equipo de infraestructura gestiona todo mediante IaC con Terraform.

Deuna! es una marca de Banco Pichincha C.A. con domicilio en Av. Japón y Alfonso Pereira, edificio Zaigen, piso 4, of 401

©2024 Todos los derechos reservados. 

Tarea 1 — Monitoreo de escalamiento de microservicios en ECS

Escenario:

El clúster ECS de Deuna! ejecuta múltiples microservicios. Cada servicio tiene su propia configuración de tareas (deseadas, mínimas y máximas). La política de escalamiento es uniforme: se crea una nueva tarea cuando el uso de CPU y/o memoria supera el **60% en promedio**. La herramienta de observabilidad principal no es CloudWatch — es una plataforma externa que ingesta CloudWatch Logs.

Entregable:

Diseña y describe con el mayor detalle posible una solución que permita **detectar, alertar y visualizar** cuando un microservicio ha alcanzado el máximo de tareas permitido.

Tu solución debe contemplar:

- El mecanismo de detección del evento de escalamiento máximo
- La integración con la herramienta de observabilidad externa
- El contenido y estructura de la alerta generada
- Cómo se evita ruido excesivo en entornos con múltiples servicios
- Una propuesta de visualización o dashboard para el equipo de guardia

Incluye diagramas de arquitectura si lo consideras necesario.

Tarea 2 — Costeo por tipo de transacción

Escenario:

Cada flujo transaccional de Deuna! (monetario y no monetario) consume un conjunto diferente de servicios AWS: Lambda, API Gateway, RDS Aurora, SQS, S3, entre otros. Cada servicio tiene su propio modelo de costeo. Los servicios son en su mayoría **compartidos** entre flujos.

Entregable:

Diseña una solución automatizada que permita **identificar, distinguir y calcular el costo aproximado por transacción** según su tipo (monetaria / no monetaria).

Tu solución debe contemplar:

- Cómo se etiqueta y propaga el tipo de transacción a través de los servicios AWS
- Cómo se atribuye el costo de servicios compartidos a transacciones individuales
- Qué herramientas de AWS usarías para el análisis de costos granular
- Cómo se automatiza el pipeline de recolección y cálculo
- Cómo presentarías los resultados a un equipo de producto o finanzas

Tarea 3 — Implementación IaC con Terraform

Entregable:

Diseña e implementa un repositorio Terraform que provisione los siguientes recursos en AWS. El repositorio debe reflejar buenas prácticas de estructura, modularización y gestión de estado.

Recursos a implementar:

a) Llave KMS

- Llave simétrica para encriptación de datos en reposo
- Key policy con least privilege
- Rotación automática habilitada
- Tags de clasificación apropiados

b) Bucket S3

- Encriptado con la llave KMS creada
- Bucket policy con mejores prácticas de seguridad para datos financieros
- Versionado habilitado
- Lifecycle policy definida
- Sin acceso público

c) Cluster RDS Aurora PostgreSQL

- Encriptado en reposo con la llave KMS
- Desplegado en subnets privadas
- Acceso restringido por security group
- Backups automatizados habilitados
- Monitoreo y métricas de performance habilitados

d) Secreto en AWS Secrets Manager

- Credenciales del usuario pgadmin (username + password)
- Encriptado con la llave KMS
- Rotación automática configurada
- Resource policy con acceso restringido

Consideraciones adicionales:

- Propón y justifica la estructura de carpetas del repositorio
- Explica cómo gestionarías el estado remoto de Terraform
- Indica qué herramientas de validación de seguridad usarías antes del apply

Tarea 4 — Escenario de incidente en producción

Escenario:

A las 2:47 AM, el sistema de alertas notifica que el servicio payments-processor en ECS ha alcanzado su máximo de tareas (10/10) y la latencia p99 supera los 4 segundos. El equipo de guardia te escala el incidente.

Entregable:

Describe cómo gestionarías este incidente desde la detección hasta el cierre. Tu respuesta debe cubrir:

- Pasos de diagnóstico y orden de ejecución
- Cómo determinas el origen del problema (capacidad, base de datos, dependencia externa)
- Acciones de mitigación inmediata sin generar downtime
- Cómo comunicas el estado a distintas audiencias (equipo técnico, producto, C-Level)
- Qué incluirías en el postmortem para prevenir recurrencia

Criterios de evaluación

Dimensión	Peso
Profundidad técnica y precisión de las soluciones	30%
Aplicación de mejores prácticas de seguridad AWS	25%
Calidad y estructura del código Terraform	20%
Razonamiento sobre costo-eficiencia	15%
Claridad y estructura de la entrega	10%